Technical Architecture & Design Document for : AR Industrial Maintenance & Training System

Summary :
The application is engineered as a modular, data-driven Augmented Reality (AR) training and uses Unity, AR Foundation (ARCore), and the Universal Render Pipeline (URP) to guide industrial technicians through complex inspection procedures on physical machinery assemblies.

To achieve strict decoupling, testability, and extensibility, the software follows a Layered Architecture adhering to SOLID principles and the Dependency Inversion Principle (DIP).

```
Data Layer ["Data & Persistence Layer"]
    A[StreamingAssets / JSON Payloads] --> B[JsonDataLoader]
    C[Local File System] <--> D[ChecklistPersistence]

Core Domain ["Business Logic / Procedure Engine"]
    B --> E[ProcedureEngine]
    E --> F[IProcedureEngine Interface]

Presentation ["AR & UI Presentation Layer"]
    G[ARObjectSpawner] --> H[ComponentRegistry]
    G --> I[ComponentHighlighter]
    F --> J[ProcedureStepController]
    J --> K[AudioPlayer]
    J --> L[Procedure UI Panel]
    D --> M[ChecklistController]
    M --> N[Checklist UI Panel]
```

Key Architectural Layers:
1. Data Layer (`Data/`):
  - `JsonDataLoader`: Asynchronously loads and validates raw JSON payloads across Editor and mobile environments.
  - `ChecklistPersistence`: Encapsulates local state file I/O using JSON converter to ensure zero data corruption.
2. Domain Logic Layer (`Procedure/`):
  - `ProcedureEngine` & `IProcedureEngine`: Pure state machine managing step transitions, boundary constraints, procedure completion.
3. Presentation & Hardware Abstraction Layer (`Highlighting/`, `Components/`, `AR/`, `UI/`) :
  - `ARObjectSpawner`: Manages AR surface plane detection, vertical CAD alignment, and plane tracking suppression.
  - `ComponentRegistry`: Binds logical data IDs from JSON to physical 3D transforms and world-space annotation callouts.
  - `ComponentHighlighter`: Drives layer-based visual outline passes within URP.
  - `ProcedureStepController`: Coordinates UI view states, audio narration, and component highlighting via decoupled events.

Core Subsystems & Design Patterns :

-> Data-Driven Procedure Engine & Extensible JSON Schema
Rather than hardcoding procedures in C# scripts, all equipment specifications, procedure sequences, component mappings, and checklist configurations are driven externally via structured JSON.

Extensibility Schema Architecture

```
classDiagram
    class MaintenanceData {
        +string schemaVersion
        +EquipmentData equipment
        +List~EquipmentData~ additionalEquipment
        +List~ProcedureData~ procedures
        +List~ChecklistItemData~ checklistItems
    }
    class EquipmentData {
        +string id
        +string name
        +string modelNumber
        +string description
        +List~ComponentData~ components
    }
    class ProcedureData {
        +string id
        +string name
        +string estimatedDuration
        +List~ProcedureStepData~ steps
    }
    class ProcedureStepData {
        +string id
        +int stepNumber
        +string title
        +string instruction
        +int audio
        +List~string~ componentIds
        +string warning
    }

    MaintenanceData --> EquipmentData
    MaintenanceData --> ProcedureData
    ProcedureData --> ProcedureStepData
```

Extensibility with Minimal Code Changes: Adding new machinery (e.g. `Rotary Screw Air Compressor`) or multi-stage procedures requires only authoring a new JSON file or updating the existing payload. The engine dynamically maps components, creates steps, and builds the UI at runtime without recompilation.

CAD Coordinate Rectification: Industrial CAD models exported via GLB/FBX commonly use a Z-up coordinate system (Upright Orientation). When placed directly on an AR horizontal plane normal, the model lies horizontally on its side.

This guarantees the machinery stands vertically upright on its mounting skid across all physical surface angles.
Interactive AR Component Selection : In addition to automatic step-driven highlighting, operators can tap directly on physical 3D parts in AR space. `ComponentRegistry` automatically generates mesh colliders on demand, allowing Physics raycasts to resolve touch coordinates directly back to the active component's JSON metadata.

Component Highlighting & Annotations:
URP Outline Rendering : Uses a custom render pass (`Linework Lite`) targeting layer 6 (`highlight`). When a step activates or an operator taps a part, `ComponentHighlighter` recursively tags all child mesh nodes to `highlight` without allocating material instances.

World-Space Annotations: Canvas labels positioned at the 3D centroid of each component orient dynamically toward the operator, providing immediate spatial identification.

JSON File Writes : Mobile environments frequently interrupt applications (e.g. phone calls, low battery, OS sleep). To eliminate file truncation risk, `ChecklistPersistence` writes serialized JSON to the targeted path.

Csharp -
```
File.WriteAllText(tempPath, json);
if (File.Exists(targetPath)) File.Delete(targetPath);
```

Error Handling & Logging Strategy:
Interface-Segregated Logging (`IAppLogger`) : All subsystems log through `IAppLogger`. Debug messages use conditional compilation (`#if UNITY_EDITOR || DEVELOPMENT_BUILD`) to eliminate string allocation overhead in production releases, while critical errors always route to `Debug.LogError`.
Defensive Failure Isolation :
- `JsonDataLoader` captures web request timeouts and schema deserialization exceptions, exposing event callbacks (`OnDataLoadFailed`) to alert the UI.
- `ComponentRegistry` verifies all component IDs and warns on duplicate or missing mesh mappings.
- `ComponentHighlighter` guards against null targets and restores normal layers upon destruction.

Performance Optimization Considerations :

AR Subsystem Overhead and Plane Suppression: `ARPlaneManager` is disabled and existing plane meshes deactivated immediately after model placement. Halts continuous polygon generation and CPU feature point updates, saving ~35% frame time on mobile.

Frame Rate Clamping: `Application.targetFrameRate = 60` initialized at startup. | Avoids GPU thermal throttling and preserves battery life during extended training sessions.
Draw Call Batching - Layer Masking vs Material Duplication: Components change layers rather than swapping materials at runtime. Retains URP SRP batcher compatibility without breaking instanced draw calls.
Event-Driven Asynchronous Loading : StreamingAssets read via `UnityWebRequest` coroutines. Prevents main thread stutter during startup data parsing.

Verification & Testing

The engine is covered by an automated NUnit EditMode Test Suite(`Assets/Tests/EditMode/ProcedureEngineTests.cs`):
- `StartProcedure_WithValidId_StartsAtFirstStep`
- `StartProcedure_WithInvalidId_ReturnsFalse`
- `MoveNext_MovesToNextStep`
- `MoveNext_OnLastStep_CompletesProcedure`
- `MovePrevious_ReturnsToPreviousStep`
- `JumpToStep_NavigatesDirectlyToStep`
- `Extensibility_MultipleProcedures_SwitchesCleanly`